

Lab #2

Group number : 5

Group members : Aditya Ray - 1233750715

Krutyanjay Shinde - 1234108189

Table of Contents

[Table of Contents](#)

[Pre - Lab](#)

[Post - LAB](#)

[Check-off Questions](#)

Pre - Lab

1. A.

The output data rate (**ODR**) is determined by a register on the IMU chip called CTRL1_XL. On the Pololu robot, the CTRL1_XL register is set to hexadecimal value 0x30 (0b00110000 in binary) during initialization of the robot (by writing to the I²C bus). What is the ODR rate that this specifies?

- Then IMU library of RP2040 sets the odr_acc value to **0b0011** which is **52Hz** according to LMS6DSO datasheet Section 9.12 Table 44

```
robot-lib > src > c imu.c > ...
1  /// @file imu.c
2  /// contains implementations for the imu library
3  #include <imu.h>
4  #include <hardware/i2c.h>
5  #include <hardware/gpio.h>
6
7
8  // Output Data Rate
9  const uint8_t odr_acc = 0b0011; // 52Hz
10 const uint8_t odr_gyro = 0b0101; // 208Hz
11
12 // Measurement Ranges
13 const uint8_t fs_acc = 0b00; // +- 2g
14 const uint8_t fs_gyro = 0b110; // +- 2000 dps
```

Table 44. Accelerometer ODR register setting

ODR_XL3	ODR_XL2	ODR_XL1	ODR_XL0	ODR selection [Hz] when XL_HM_MODE = 1 in CTRL6_C (15h)	ODR selection [Hz] when XL_HM_MODE = 0 in CTRL6_C (15h)
0	0	0	0	Power-down	Power-down
1	0	1	1	1.6 Hz (low power only)	12.5 Hz (high performance)
0	0	0	1	12.5 Hz (low power)	12.5 Hz (high performance)
0	0	1	0	26 Hz (low power)	26 Hz (high performance)
0	0	1	1	52 Hz (low power)	52 Hz (high performance)
0	1	0	0	104 Hz (normal mode)	104 Hz (high performance)
0	1	0	1	208 Hz (normal mode)	208 Hz (high performance)
0	1	1	0	416 Hz (high performance)	416 Hz (high performance)
0	1	1	1	833 Hz (high performance)	833 Hz (high performance)
1	0	0	0	1.66 kHz (high performance)	1.66 kHz (high performance)
1	0	0	1	3.33 kHz (high performance)	3.33 kHz (high performance)
1	0	1	0	6.66 kHz (high performance)	6.66 kHz (high performance)
1	1	x	x	Not allowed	Not allowed

- ODR is not dependent on the power mode of the accelerometer, but power mode is dependent on ODR. if an ODR is selected which does not match the configuration in CTRL6_C register then CTRL6_C is ignored.

B.

The accelerometer on the robot can be configured to have one of four **ranges** by setting the CTRL1_XL register. Each range is a multiple of g , the acceleration due to gravity at the earth's surface, defined to be 9.80665 m/s^2 . What are the ranges supported by the accelerometer on the IMU chip?

- As seen in the Table we can set the ranges of **2g, 4g, 8g, 16g**.

Table 45. Accelerometer full-scale selection

FS[1:0]_XL	XL_FS_MODE = '0' in CTRL8_XL (17h)	XL_FS_MODE = '1' in CTRL8_XL (17h)
00 (default)	2 g	2 g
01	16 g	2 g
10	4 g	4 g
11	8 g	8 g

C.

The robot software configures the accelerometer hardware to use one of these four ranges during initialization. What is this range?

- The IMU library of RP2040 set the range of the accelerometer to be **2g**.

```

robot-lib > src > c imu.c > [?] fs_acc
1  /// @file imu.c
2  /// contains implementations for the imu library
3  #include <imu.h>
4  #include <hardware/i2c.h>
5  #include <hardware/gpio.h>
6
7
8  // Output Data Rate
9  const uint8_t odr_acc = 0b0011; // 52Hz
10 const uint8_t odr_gyro = 0b0101; // 208Hz
11
12 // Measurement Ranges
13 const uint8_t fs_acc = 0b00; // +- 2g
14 const uint8_t fs_gyro = 0b110; // +- 2000 dps
15

```

D.

The IMU chip offers an optional low pass filter (**LPF**) that performs additional smoothing of the signal from the accelerometer at the cost of additional latency. By default, the robot initializes the IMU to not use this filter. What value would you set the CTRL1_XL register to to use the filter and leave the ODR and range as above?

- **0x32**. It's derived by combining the binary values for the ODR (0011), full-scale range (00), and the LPF enable bit (1), resulting in 0b00110010.

2. A.

According to table 2 of the [inertial module datasheet](#), with the default range, the **sensitivity** of the accelerometer is 0.061 mg/LSB. Here, mg stands for milli-g's and **LSB** for least significant bit. The accelerometer has a 16-bit analog to digital converter (**ADC**). Reading its raw data over the I²C bus yields a 16-bit signed integer. Explain where the number 0.061 comes from.

- The accelerometer's sensitivity is derived by distributing its total measurement range across the full resolution of the 16-bit Analog-to-Digital Converter (ADC).
1. **Total Range:** The default full-scale setting is **±2 g**, which is a total measurement span of **4 g**.
 2. **ADC Resolution:** A 16-bit ADC can represent **65,536** distinct digital values (2¹⁶). Each value is one Least Significant Bit (LSB).
 3. **Sensitivity:** To find the value of a single LSB, the total range is divided by the number of digital values:

$$65,536 \text{ LSBs} / 4 \text{ g} \approx \mathbf{0.000061 \text{ g/LSB}}$$

Converting this to milli-g's (mg) by multiplying by 1000 gives the final sensitivity, which matches the value in the datasheet:

$$0.000061\text{g/LSB} \times 1000 = \mathbf{0.061\text{mg/LSB}}$$

LA_So	Linear acceleration sensitivity ⁽²⁾	FS = $\pm 2\text{ g}$	± 2000	0.061	mg/LSB
		FS = $\pm 4\text{ g}$		0.122	
		FS = $\pm 8\text{ g}$		0.244	
		FS = $\pm 16\text{ g}$		0.488	

B.

As explained in Chapter 7, Sensors and Actuators, of [Lee and Seshia](#), a sensor output may be modeled by an affine function $f(x) = ax + b$, where a is the sensitivity and b is the bias. For this IMU, ideally, if x is the 16 bit integer received over the I²C bus, $a = 0.000061$, and $b = 0$, then $f(x)$ is the acceleration in g's. But both a and b may vary with temperature. How much variation should you expect (in percent per degree centigrade)? How much bias can you expect (in mg)? How much should you expect the bias to change with temperature (in mg per degree centigrade)?

Hint: Table 2 could be helpful. Bias is referred to as "zero-g level" in this table.

- The linear acceleration sensitivity is $\pm 0.01\text{ \%/}^\circ\text{C}$
- The acceleration zero-g level offset accuracy is $\pm 20\text{ \%/}^\circ\text{C}$

G_So%	Sensitivity tolerance ⁽³⁾	at component level	± 1	%
LA_SoDr	Linear acceleration sensitivity change vs. temperature ⁽⁴⁾	from -40° to $+85^\circ$	± 0.01	$\text{\%/}^\circ\text{C}$
G_SoDr	Angular rate sensitivity change vs. temperature ⁽⁴⁾	from -40° to $+85^\circ$	± 0.007	$\text{\%/}^\circ\text{C}$
LA_TyOff	Linear acceleration zero-g level offset accuracy ⁽⁵⁾		± 20	mg
G_TyOff	Angular rate zero-rate level ⁽⁵⁾		± 1	dps
LA_OffDr	Linear acceleration zero-g level change vs. temperature ⁽⁴⁾		± 0.1	$\text{mg/}^\circ\text{C}$
G_OffDr	Angular rate typical zero-rate level change vs. temperature ⁽⁴⁾		± 0.010	$\text{dps/}^\circ\text{C}$
Rn	Rate noise density in high-performance mode ⁽⁶⁾		3.8	$\text{mdps}/\sqrt{\text{Hz}}$
RnRMS	Gyroscope RMS noise in normal/low-power mode ⁽⁷⁾		75	mdps

Post - LAB

1. Suppose a sensor gives you a 16-bit signed integer representing some measured quantity in some well-defined units, that the range of the sensor in those units is -4.0 to 4.0, and that the sensitivity in those units is 0.000122 ($1/2^{13}$). What is representation in hex and in binary of a sensor reading of 1.0 in those units?
 - The correct hexadecimal representation for a sensor reading of 1.0 is **0x2000**, and the binary representation is **0b0010 0000 0000 0000**.
 - 1. **Positive Range & Resolution:** A 16-bit signed integer uses 15 bits for positive values, which provides 2¹⁵ or **32,768** digital steps (LSBs). The sensor's positive measurement range is **4.0 units**.
 - 2. **Steps per Unit:** To find how many digital steps represent one unit, divide the total positive steps by the total positive range:
 $4.0 \text{ units} / 32,768 \text{ LSBs} = 8192 \text{ LSBs/unit}$
 - 3. **Decimal Value of Reading:** For a reading of 1.0 unit, the corresponding decimal ADC value is:
 $1.0 \text{ unit} \times 8192 \text{ LSBs/unit} = 8192$
 - 4. **Unit Conversion:** The decimal value 8192 is converted to hexadecimal and binary:
 - o **Hexadecimal: 0x2000**
 - o **16-bit Binary: 0010 0000 0000 0000**

2. Suppose that your goal with the accelerometer is to measure the tilt in degrees of the surface on which your robot is moving. How would you measure the bias and then compensate for this bias?
 1. **Calibration Setup:** Place the robot on a surface that seems to be reasonably flat. The accelerometer's Z-axis should be pointing directly up, perpendicular to the surface.
 2. **Data Acquisition:** While the robot is completely stationary, take some readings from the accelerometer and **calculate the average** for each of the X, Y, and Z axes.
 3. **Bias Calculation:** The ideal reading on a perfectly level surface is (X=0, Y=0, Z=+1g). The bias is the difference between your average measured reading and this ideal value.
 - This also gets rid of the zero-g inbuilt bias as each reading will have that bias counted in every reading. (assuming your temperature remains same along different readings)
 - o **Bias_X = Average_X_Reading - 0**
 - o **Bias_Y = Average_Y_Reading - 0**

$$\text{Bias_Z} = \text{Average_Z_Reading} - 1.0$$

4. **Compensation:** Store this calculated bias vector (**Bias_X**, **Bias_Y**, **Bias_Z**). For all subsequent measurements, **subtract this stored bias** from the new raw readings to get the corrected value:

$$\begin{aligned}\text{Corrected_X} &= \text{Raw_X} - \text{Bias_X} \\ \text{Corrected_Y} &= \text{Raw_Y} - \text{Bias_Y} \\ \text{Corrected_Z} &= \text{Raw_Z} - \text{Bias_Z}\end{aligned}$$

3. Suppose that your goal is to orient the robot to point straight up the hill. How would you use the pitch and roll measurements to accomplish this?
 - To orient the robot to point straight up a hill, we must adjust its orientation until its **roll is zero** and its **pitch is maximized** (max in our cause is **$2g \sim x = (2^{15}) - 1 = 32767$**)
 - We choose a direction either left or right and keep rotating the robot in such a way that we get roll =0 and pitch being maximum positive number.

4. What were your takeaways from the lab? What did you learn during the lab? Did any results in the lab surprise you?

- We learned how to configure the IMU, specifically the accelerometer, by setting its register values.
- Reading the datasheet we got to know about additional features and configuration options, such as the built-in low-pass filter and various power modes.
- Most importantly, we learned how to interpret the raw accelerometer data to determine our robot's orientation.
- The most surprising observation was how temperature can affect the sensor's readings.
 - The datasheet quantifies how both the **sensitivity** and the **in-built bias** (zero-g offset) change with temperature.
 - We think this might be critical for applications where precision is essential. For example, a drone flying to a higher altitude will experience a drop in temperature. If this change in readings from the sensor isn't accounted for, the flight controller might misinterpret the drone's actual orientation, leading to instability in PID or which every control algorithm used.

Check-off Questions

1. Interpreting the numbers

A. Explain why, when the robot is sitting on a flat surface, the sensed accelerations in the x and y directions are near zero and in the z direction near one.

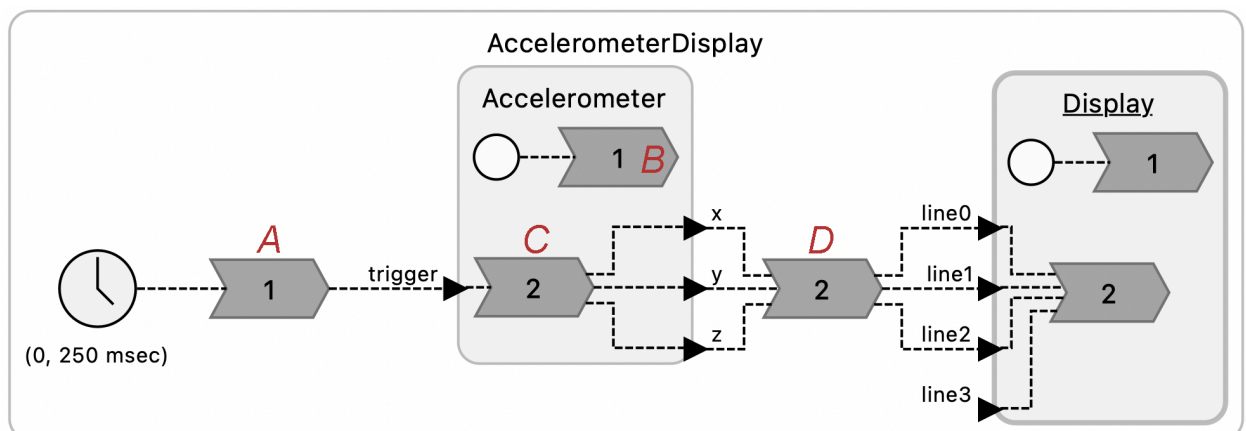
- When the robot sits on a flat, level surface, it is not in freefall. The surface pushes up on the robot with a force equal to the force of gravity. The accelerometer's Z-axis, pointing down from the plane of the bot, measures this constant downward push. Hence we see **1g** in the display.
- Because the surface is perfectly level and the bot is still, the force of gravity being the only force aligns perfectly with the z-axis and perpendicular to the accelerometer's X and Y axes. Therefore, there is no component of the gravitational force acting along these horizontal directions. Since the robot is stationary and not accelerating sideways, the X and Y axes **0 g**.

B. Why is the z direction not near the negative one? Doesn't gravitation pull you down, not up?

- This depends on the **frame of reference**. Depending on which direction the sensor considers positive or negative we get the respective readings. The data in the **sensor is stored in 2's complement form** thus if we store it in a signed 16 bit integer it will store a negative integer for negative readings and a positive for positive readings. Since the z - axis shows positive when kept upright on a flat surface we can say that the sensor considers **downward direction as positive**.

2.

- a. Open the file src/AccelerometerDisplay.lf in VS Code. Enable the diagram, navigate and read the code, and explain what each of the reactions labeled A, B, C, and D does



- So the four reactions are for controlling the data flow.
- Reaction A is a timer that triggers every 250ms to read data periodically.
- B is a startup reaction that runs once at the beginning to initialize the IMU (This reaction is inherited from the IMU reactor).
- The trigger from A makes reaction C read the x, y, and z values in g (acceleration due to gravity) from the hardware.
- Finally, reaction D takes those x, y and z values, formats them, stores them into a string buffer and to be sent to the respective line ports in display.

b. Notice the target specification at the top of each of the .lf files. What do you think is the significance of the directive:

single-threaded: true.

- The **single-threaded: true** ensures the reactor doesn't run concurrently. The i2c

```
robot-lib > src > C imu.c > imu_read_acc(imu_inst_t*, axes_data_t*)
86 void _imu_set() {
103     // write to CTRL2_G
104     cmd_gyro_buf[1] = (odr_gyro << 4 | fs_gyro << 1) | 0b0;
105     i2c_write_blocking(i2c0, _IMU_ADDR, cmd_gyro_buf, 2, false);
106 }
107
108 uint imu_read_acc(imu_inst_t* imu_inst, axes_data_t* acc_data) {
109     if (!imu_inst->status) {
110         return -1;
111     }
112     uint8_t reg = _IMU_OUTX_L_XL;
113     uint8_t buf[6];
114     int16_t raw[3];
115     // read from the register
116     i2c_write_blocking(i2c0, _IMU_ADDR, &reg, 1, true);
117     i2c_read_blocking(i2c0, _IMU_ADDR, buf, 6, false);
118     /// TODO: might be other way
119     for (int i = 0; i < 3; i++) {
120         raw[i] = (buf[(i * 2) + 1] << 8 | buf[i * 2]);
121     }
122     acc_data->x = (raw[0] * sen_acc) / 1000.0;
123     acc_data->y = (raw[1] * sen_acc) / 1000.0;
124     acc_data->z = (raw[2] * sen_acc) / 1000.0;
125     return 0;
126 }
```

read and writes use the blocking functions which will block the CPU until the data is written or read completely.

Ref: <https://www.lf-lang.org/docs/reference/target-declaration>

